

The AI Delivery Maturity Curve

LEVEL 1

AI as Assistant
(Chat help)



LEVEL 2

AI as Accelerator
(Code generation)



LEVEL 3

AI as Collaborator
(Spec-driven workflows)



LEVEL 4

AI as Translation Layer
(Business → Technical intent)



LEVEL 5

AI as Delivery System Partner
(Continuous adaptive SDLC)

Higher maturity requires:

- stronger context
- stronger architecture governance
- stronger review discipline
- stronger system observability

The experiment is not:

"Can AI write code?"

The experiment is:

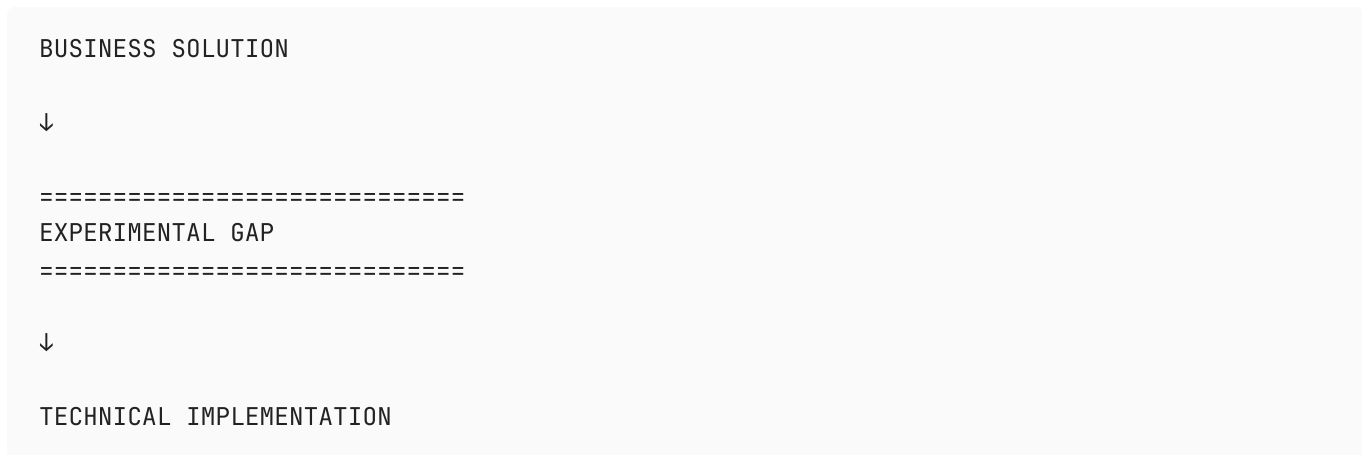
"Can AI reliably participate in software delivery as a context-aware engineering collaborator?"

The AI Adoption Journey Across the SDLC

Phase	Goal	Human Role	AI Role
1	Increase context quality and reduce onboarding friction	Knowledge structuring and validation	Summarization and synthesis
2	Accelerate implementation work while preserving quality	Planning and review	Code generation and augmentation
3	Test whether AI can bridge analysis → implementation	Constraint definition and architectural governance	Specification transformation
4	Create a continuously improving AI-enabled SDLC		

The Critical Unknown

“Can AI meaningfully compress the translation layer between business intent and technical execution?”



Can AI reliably infer:

- Existing architecture constraints?
- Service boundaries?
- Ownership?
- Cross-system impacts?
- Data flow implications?
- Brownfield conventions?
- Non-functional requirements?
- Deployment implications?
- Hidden tribal knowledge?

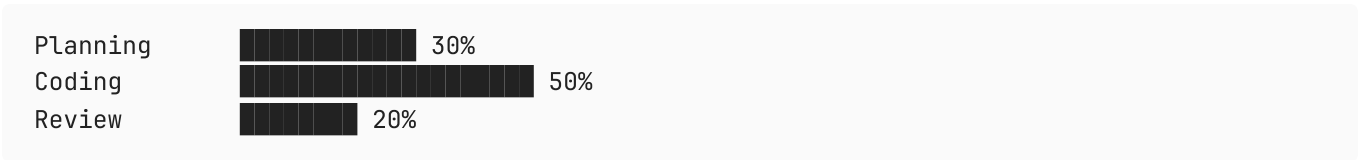
Steering files reduce ambiguity,
but may not fully replace:

- architectural intuition
- system history
- contextual tradeoffs
- institutional knowledge

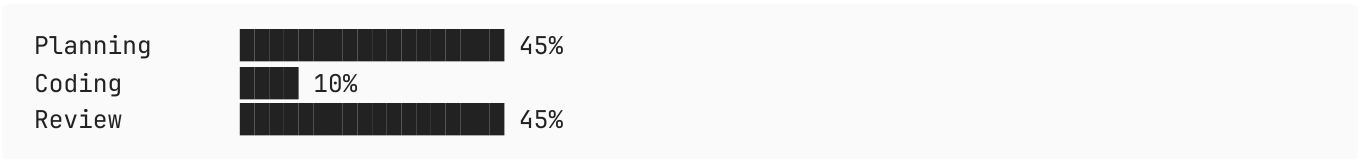
Additional Thoughts

Developer Role Shift

Traditional Development



AI-Assisted Development



And then the key insight underneath:

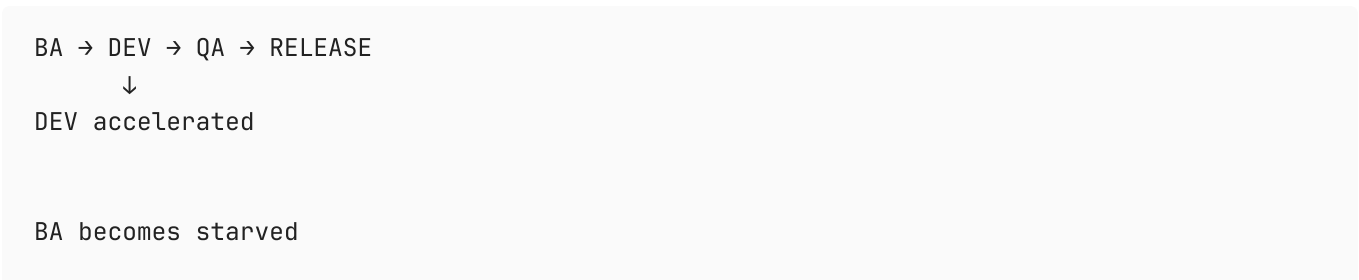
AI reduces typing effort.It does NOT reduce accountability.Engineering value shifts from:"writing code"toward:- defining intent- constraining solutions- validating correctness- reviewing generated output- architectural decision-making

That’s an extremely important cultural message for senior engineers.

Value Stream Bottleneck

Future Risk State

(if development is truly the bottleneck)



OR
QA becomes overloaded
OR
Both

If Development is not the bottleneck

- Other queues & droughts may appear

Based on what early adopters are seeing:

- AI code making its way to production is low
- Code review is a new bottleneck
- Releases will become larger and riskier with even more administration, training, support and PIT requirements

Opportunities for QA

Traditional QA

Manual test case derivationManual regression identificationHuman-heavy validationLate-stage involvement

↓

AI-Augmented QA

Requirements-to-test synthesisTraceability validationRegression impact analysisRisk hotspot identificationTest data generationSpecification consistency checking

And then:

The future QA role becomes:- validating business intent- evaluating edge cases- assessing risk- governing confidence rather than only executing tests